CPSC 335 | ALGORITHMS & DATA STRUCTURES

# RescueRoute

## Surplus Food Matching & Pickup Planner

An explainable Python application comparing FIFO and urgency-first Greedy scheduling.




Simulated prototype

John • Ian • Andrew

WHY IT MATTERS

# The problem: good food, limited time

### Time-sensitive donations

Restaurants, grocery stores and campus events may have surplus food with a short pickup window.

### Recipient constraints

Shelters and food pantries differ in accepted food types, remaining capacity and closing times.

### Volunteer constraints

Drivers have limited availability, vehicle space, locations and pickup counts.

**Central question: How do we assign donations efficiently, transparently and explainably?**

# Our solution: an explainable matching pipeline

**1 Load CSV data**

Donations, recipients and volunteers

→

**2 Validate inputs**

IDs, types, quantities, time and capacity

→

**3 Schedule**

FIFO and urgency-first Greedy on the same data

→

**4 Explain & compare**

Human-readable decisions and eight metrics

Python command-line application

Simulated input data

# Data and matching rules

## Donations

Donation ID • donor • food type • quantity • ready and expiry times • pickup area

## Recipients

Accepted foods • available capacity • area • closing time

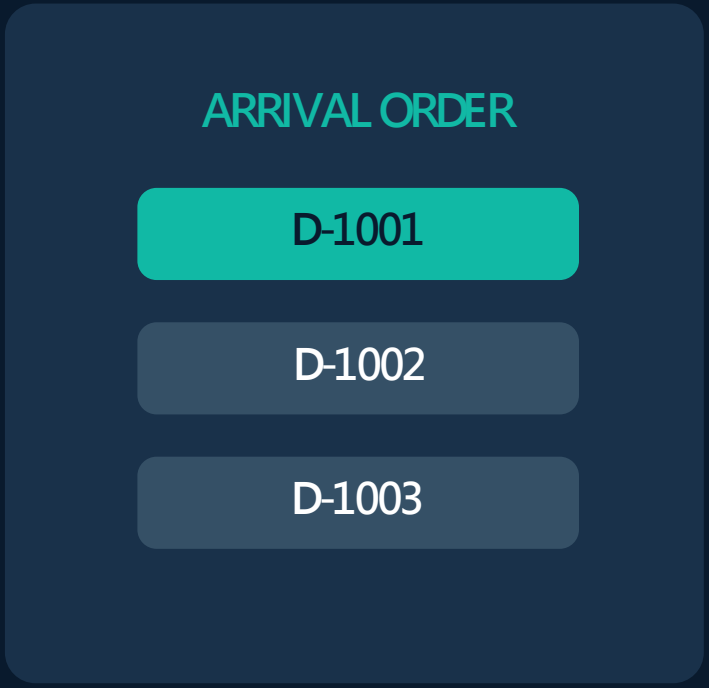
## Volunteers

Vehicle capacity • service area • availability • maximum pickups

**A match is only feasible when compatible resources and scheduling constraints allow it.**

# FIFO: First-In, First-Out

- Process donations in their input/arrival order.
- For each donation, choose the first feasible recipient and volunteer.
- If no feasible match exists, record the donation as unassigned.
- Explain each decision in plain language.




Strength: predictable • Trade-off ignores urgency

# Greedy: urgency-first scheduling

## Priority = least remaining shelf life first

Remaining shelf life = expiry_time − ready_time

### First

Shortest shelf life

### Tie-breaker

Larger food quantity

### Then

Earlier ready time, then input order

Implemented using Python's heapq priority queue.

MEASURED COMPARISON

# FIFO vs. Greedy: sample results

| METRIC | FIFO | GREEDY |
|---|---|---|
| Donations received | 8 | 8 |
| Donations assigned | 7 | 7 |
| Food quantity rescued | 170 | 170 |
| Food quantity at risk | 100 | 100 |
| Urgent rescue rate | 100% | 100% |
| Recipient utilization | 68.0% | 68.0% |
| Volunteer utilization | 27.4% | 27.4% |

**FIFO and Greedy produced the same overall metrics on this dataset, but Greedy prioritizes shorter remaining shelf life, which can lead to different assignments when resources are limited.**

# Explainable decisions and trade-offs

## Assigned

Why the food type, recipient capacity and volunteer availability made the match feasible.

## Unassigned

Why no valid recipient–volunteer combination could be found.

## Deferred

Why a more urgent donation received priority under Greedy.

## Fairness trade-off

Prioritizing expiring food can delay less urgent donations; no global optimum is guaranteed.

# Tested, validated, and transparent

**29 / 29**

**AUTOMATED TESTS PASSED**

Python unittest

**Coverage includes:**

- Valid and invalid donations
- Food-type, capacity, area and time constraints
- Empty inputs and competing donations
- FIFO/Greedy differences and deferrals

**Limitations: simulated data, no live routing, no guaranteed optimal solution.**

# What we accomplished

### John
Algorithm & Optimization Lead

Integrated FIFO and Greedy into the CLI, displayed comparison metrics, tested and documented the application.

### Ian
Data & Matching Lead

Improved food-type and area validation, reviewed matching logic and added five automated tests.

### Andrew
Product & Quality Lead

Built the comparison explanation tool and completed the FIFO-versus-Greedy report.

**RescueRoute makes algorithmic choices testable, measurable, and understandable.**

Educational prototype — not for real-world food safety or emergency deployment.